Name-Aditi Sudhir Igade.

Roll No-221081

Batch-A3

Assignment No - 4

AIM: A customer wants to travel from source A to source B. He books a cab from source A to reach destination B. Calculate the shortest path by avoiding real time traffic to reach destination B.

OBJECTIVE:

- To understand the concept and representation of graphs
- To implement the Dijkstra's algorithm in Java programming language

THEORY:

Dijkstra’s Algorithm:

Dijkstra's algorithm allows us to find the shortest path between any two vertices of a graph. It differs from the minimum spanning tree because the shortest distance between two vertices might not include all the vertices of the graph.

ALGORITHM:

1. Create a set of visited vertices, initially empty.
2. Create a priority queue of vertices to explore, initially containing only the source vertex. The tentative distance from the source vertex to itself is 0, and the distance to all other vertices is infinity.
3. While the priority queue is not empty:
    a) Remove the vertex with the smallest tentative distance from the priority queue. Call it the current vertex.
    b) If the current vertex is the destination vertex, stop. The shortest path has been found.
    c) If the current vertex has already been visited, skip it.
    d) Mark the current vertex as visited.
    e) For each neighbor of the current vertex that has not been visited:
        - Calculate the tentative distance to the neighbor as the sum of the current vertex's tentative distance and the weight of the edge between them.

    - If the tentative distance is smaller than the neighbor's current tentative distance, update the neighbor's tentative distance and set its predecessor to the current vertex.
    - Add the neighbor to the priority queue.
4. If the destination vertex was not reached, there is no path from the source vertex to the destination vertex.
5. To get the shortest path, start at the destination vertex and follow the predecessors back to the source vertex.

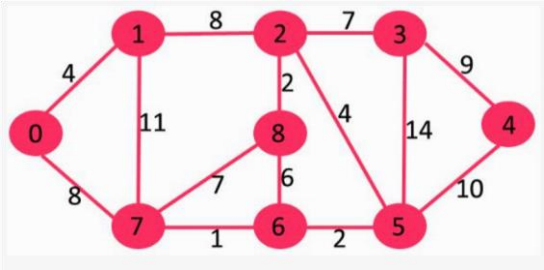
CODE:

```
import java.util.*;


public class Main{
    static Scanner sc = new Scanner(System.in);
    static int V;
    int minDist(int dist[], Boolean sptSet[]){
        int min = Integer.MAX_VALUE, min_index = -1;
        for (int v = 0; v < V; v++){
            if (sptSet[v] == false && dist[v] <= min){
                min = dist[v];
                min_index = v;
            }
        }
        return min_index;
    }

    void printSolution(int dist[]){
        System.out.println("Vertex\t\tDistance from source");
        for (int i = 0; i < V; i++){
            System.out.println(i + "\t\t" + dist[i]);
        }
    }

    void djikstra(int graph[][], int src){
        int dist[] = new int[V];
        Boolean sptSet[] = new Boolean[V];
        for (int i = 0; i < V; i++){
            dist[i] = Integer.MAX_VALUE;
            sptSet[i] = false;
        }

        dist[src] = 0;
        for (int c = 0; c < V-1; c++){
            int u = minDist(dist, sptSet);
            sptSet[u] = true;
            for (int v = 0; v < V; v++){
                if (!sptSet[v] && graph[u][v] != 0 && dist[u] !=
Integer.MAX_VALUE && dist[u] + graph[u][v] < dist[v]){
                    dist[v] = dist[u] + graph[u][v];
                }
            }
        }
        printSolution(dist);
    }
```

```java
    public static void main(String[] args){

        System.out.println("Enter number of vertices: ");
        V = sc.nextInt();
        int graph[][] = new int[V][V];
        for (int i = 0; i<V; i++){
            for (int j = 0; j<V; j++){
                graph[i][j] =0;
            }
        }
        int ele;
        System.out.println("Enter graph in form of matrix: ");
        for (int i = 0; i < V; i++){
            for (int j = 0; j < V; j++){
                System.out.print("Enter weight from "+i+" to "+j+": ");
                graph[i][j] = sc.nextInt();
            }
        }
        Main obj = new Main();
        obj.djikstra(graph, 0);
    }
}
```

INPUT/OUTPUT:

Input graph:




```
Enter number of vertices:
9
Enter graph in form of matrix:
Enter weight from 0 to 0: 0
Enter weight from 0 to 1: 4
Enter weight from 0 to 2: 0
Enter weight from 0 to 3: 0
Enter weight from 0 to 4: 0
Enter weight from 0 to 5: 0
Enter weight from 0 to 6: 0
Enter weight from 0 to 7: 8
Enter weight from 0 to 8: 0
Enter weight from 1 to 0: 4
Enter weight from 1 to 1: 0
Enter weight from 1 to 2: 8
Enter weight from 1 to 3: 0
Enter weight from 1 to 4: 0
Enter weight from 1 to 5: 0
Enter weight from 1 to 6: 0
Enter weight from 1 to 7: 11
Enter weight from 1 to 8: 0
Enter weight from 2 to 0: 0
Enter weight from 2 to 1: 8
Enter weight from 2 to 2: 0
Enter weight from 2 to 3: 7
Enter weight from 2 to 4: 0
Enter weight from 2 to 5: 4
Enter weight from 2 to 6: 0
Enter weight from 2 to 7: 0
Enter weight from 2 to 8: 2
Enter weight from 3 to 0: 0
Enter weight from 3 to 1: 0
Enter weight from 3 to 2: 7
Enter weight from 3 to 3: 0
Enter weight from 3 to 4: 9
```

```
Enter weight from 3 to 5: 4
Enter weight from 3 to 6: 0
Enter weight from 3 to 7: 0
Enter weight from 3 to 8: 2
Enter weight from 4 to 0: 0
Enter weight from 4 to 1: 0
Enter weight from 4 to 2: 0
Enter weight from 4 to 3: 9
Enter weight from 4 to 4: 0
Enter weight from 4 to 5: 10
Enter weight from 4 to 6: 0
Enter weight from 4 to 7: 0
Enter weight from 4 to 8: 0
Enter weight from 5 to 0: 0
Enter weight from 5 to 1: 0
Enter weight from 5 to 2: 4
Enter weight from 5 to 3: 14
Enter weight from 5 to 4: 10
Enter weight from 5 to 5: 0
Enter weight from 5 to 6: 2
Enter weight from 5 to 7: 0
Enter weight from 5 to 8: 0
Enter weight from 6 to 0: 0
Enter weight from 6 to 1: 0
Enter weight from 6 to 2: 0
Enter weight from 6 to 3: 0
Enter weight from 6 to 4: 0
Enter weight from 6 to 5: 2
Enter weight from 6 to 6: 0
Enter weight from 6 to 7: 1
Enter weight from 6 to 8: 6
Enter weight from 7 to 0: 8
Enter weight from 7 to 1: 11
Enter weight from 7 to 2: 0
Enter weight from 7 to 3: 0
```

```
Enter weight from 7 to 4: 0
Enter weight from 7 to 5: 0
Enter weight from 7 to 6: 1
Enter weight from 7 to 7: 0
Enter weight from 7 to 8: 7
Enter weight from 8 to 0: 0
Enter weight from 8 to 1: 0
Enter weight from 8 to 2: 2
Enter weight from 8 to 3: 0
Enter weight from 8 to 4: 0
Enter weight from 8 to 5: 0
Enter weight from 8 to 6: 6
Enter weight from 8 to 7: 7
Enter weight from 8 to 8: 0
Vertex          Distance from source
0               0
1               4
2               12
3               19
4               21
5               11
6               9
7               8
8               14
```

CONCLUSION:

We learnt to implement graph data structure in JAVA programming language. We applied the Djikrtra's algorithm to find the shortest path between 2 nodes.